

Table of Contents

1. What is the business problem that you are attempting to solve?	2
Why is Python a good way to solve this problem?	2
2. Describe the Dataset	2
2.1 How many variables are in the data set?	2
2.2 What are the data types and levels of measurement for each of the variables?	3
2.3 How many observations are in the data set?	4
2.4 Are there any missing values or outliers in the data set?	4
2.5 How did you choose to handle the missing values and outliers?	5
2.6 What are some of the descriptive statistics for key variables?	6
4.1 What did you learn about the data?	12
4.2 Why is or isn't Python a good tool to use for this type of analysis?	13
References	14

1. What is the business problem that you are attempting to solve?

The Business is a UK based non-store online retail brand which sells all-occasions gifts. The problem that I'm attempting to solve is finding insights of product sales, country-based sales, customer behavior for advertising campaign and to increase overall sales. The business is experiencing inconsistent sales patterns and unclear product performance trends across different countries and customer segments. There is also limited visibility into customer return behavior, which may be affecting overall profitability. The goal is to perform a comprehensive descriptive analysis to uncover:

- Top- and least-performing products.
- Sales trends over time (by month)
- Customer return patterns
- Geographic performance by country

By identifying which products sell best (and worst), when customers are most active, and which items or customers are driving high return rates, the company can optimize inventory, improve customer targeting, and reduce product return losses.

Why is Python a good way to solve this problem?

Python is an ideal program for data analytics since it has powerful libraries (Pandas, NumPy, Matplotlib, Seaborn) for data cleaning, exploration, visualization, and basic statistical analysis in a single package. It is fast, it can be customized, and makes it easy to create clean reports. Python is an excellent tool for this analysis since it has powerful data manipulation libraries like pandas and NumPy. It supports complete visualizations with matplotlib and seaborn. It allows fast cleaning, converting, and concatenation of substantial amounts of data. It can automatically perform tedious activities such as weekly best-seller products. The flexibility and scalability of Python are the features that make it a convenient and powerful technique to unlock useful insights from business data of everyday life.

2. Describe the Dataset

2.1 How many variables are in the data set?

There are 8 variables in the dataset. The column names and details of the variables given below in the table:

Variable Name	Role	Type	Description	Missing Values
InvoiceNo	ID	Categorical	a 6-digit integral number uniquely assigned to each transaction. If this code starts with the letter 'c', it indicates a cancellation	no
StockCode	ID	Categorical	a 5-digit integral number uniquely assigned to each distinct product	no
Description	Feature	Categorical	product name	yes
Quantity	Feature	Integer	the quantities of each product (item) per transaction	no
InvoiceDate	Feature	Date	the day and time when each transaction was generated	no
UnitPrice	Feature	Continuous	product price per unit	no
CustomerID	Feature	Categorical	a 5-digit integral number uniquely assigned to each customer	Yes
Country	Feature	Categorical	the name of the country where each customer resides	no

2.2 What are the data types and levels of measurement for each of the variables?

info() is used to describe a data set. Here I used df.info() to get the information about the data set and got the level of measurement as well.

Variable	Data Type	Level of Measurement	Explanation
InvoiceNo	object	Nominal	Acts as a unique ID for each invoice; no numerical meaning
StockCode	object	Nominal	Categorical identifier for items
Description	object	Nominal	Name of the product, used for grouping but not ordered
Quantity	int64	Ratio	Quantitative; has a true zero and negative values (for returns)
InvoiceDate	datetime64	Interval	Time-based; not ratio (e.g., no true zero)
UnitPrice	float64	Ratio	Continuous variable; can be compared and divided
CustomerID	float64	Nominal	Identifies the customer; not a numerical value
Country	object	Nominal	Categorical variable for customer location

2.3 How many observations are in the data set?

The Original observations of the data set is 541909. Using `head()` helps to check the first few rows of the data set so that we can review that the data set is loaded ok. After using `df.head()`, here are the results:

```
InvoiceNo StockCode ... CustomerID    Country
0  536365  85123A ...  17850.0 United Kingdom
1  536365   71053 ...  17850.0 United Kingdom
2  536365  84406B ...  17850.0 United Kingdom
3  536365  84029G ...  17850.0 United Kingdom
4  536365  84029E ...  17850.0 United Kingdom
[5 rows x 8 columns]
```

2.4 Are there any missing values or outliers in the data set?

To check the missing values, I have used `isna()` method. By using the method `df.isna().sum()`, I have there are 1454 missing values in description and 135080 missing values in CustomerID.

To check the outliers, I calculated the skewness and kurtosis, and I calculated standard deviation as well. Both results showed that the quantity and UnitPrice columns have outliers.

Python code to identify outliers-

```
#Calculating skewness and Kurtosis for Quantity

import SciPy. Stats as stats

print('Skewness=', round(df.Quantity.skew(),2))
print('Kurtosis=', round(df.Quantity.kurt(),2))
```

Result

Skewness=-0.26 Means quantity values are symmetric around the mean.

Kurtosis= 119769.16 Large quantity purchase and massive negative values from returns or data entry errors

```
#Calculating skewness and Kurtosis for Quantity

print('Skewness=', round(df.UnitPrice.skew(),2))
print('Kurtosis=', round(df.UnitPrice.kurt(),2))
```

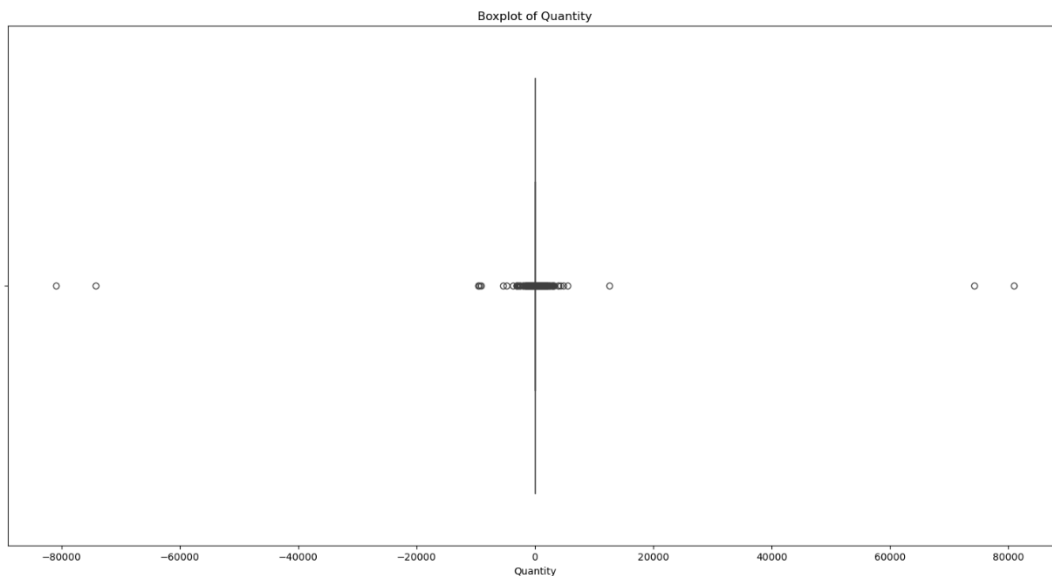
Result

Skewness= 186.51 Means most products are low prices and small number of items have high price

Kurtosis= 59005.72 Means that the data has invalid or unusual prices.

```
#Calculating P value of Skewness & Kurtosis
```

```
print('Skewness test p-  
value:',round(stats.skewtest(df.Quantity).pvalue, 4))  
print('Kurtosis test p-  
value:',round(stats.kurtosistest(df.Quantity).pvalue, 4))
```



Python code to show the outlier using boxplot.

```
#Visualizing Outliers using seaborn  
  
import seaborn as sns  
  
sns.boxplot(x=df['Quantity'])  
plt.title("Boxplot of Quantity")  
Pl. Show()
```

2.5 How did you choose to handle the missing values and outliers?

Handling Missing Values:

The dataset contained missing values primarily in the CustomerID and Description columns.

- **CustomerID:** This column had a large number of missing values (135080 rows). Since customer-related analysis is important for understanding repeat purchases or segmentation, we chose to remove rows with missing CustomerID to ensure accurate insights.
- **Description:** A few rows had missing product descriptions. Rather than dropping these, we filled them with "Unknown", as the rest of the transaction data (e.g., quantity and price) remained useful.

Python code to Handle missing values-

```
#Deleting Original InvoiceDate column
df_clean.drop(columns=['InvoiceDate'], inplace=True)

# Fill missing descriptions with 'Unknown'
df_clean['Description'] = df_clean['Description'].fillna('Unknown')

# Drop rows with missing CustomerID
df_clean.dropna(subset=['CustomerID'], inplace=True)
```

Handling Outliers:

Outliers were present in the Quantity and UnitPrice columns, confirmed through both summary statistics and visualizations (e.g., histograms, boxplots):

Negative Quantities: These usually represent product returns. We excluded them from the main sales analysis to focus on successful transactions. However, they could be separately analyzed to understand return behavior.

UnitPrice = 0 or extremely high values: Rows with a UnitPrice of zero were excluded, as they likely indicate free samples, test entries, or data errors. Similarly, entries with extremely high prices were considered outliers and removed to avoid distortion in average sales calculations.

Python code to handle outliers-

```
# Remove Quantity =0
df_clean = df_clean[df_clean['UnitPrice'] > 0]

# Remove negative Quantity (returns)
df_clean = df_clean[df_clean['Quantity'] > 0]
```

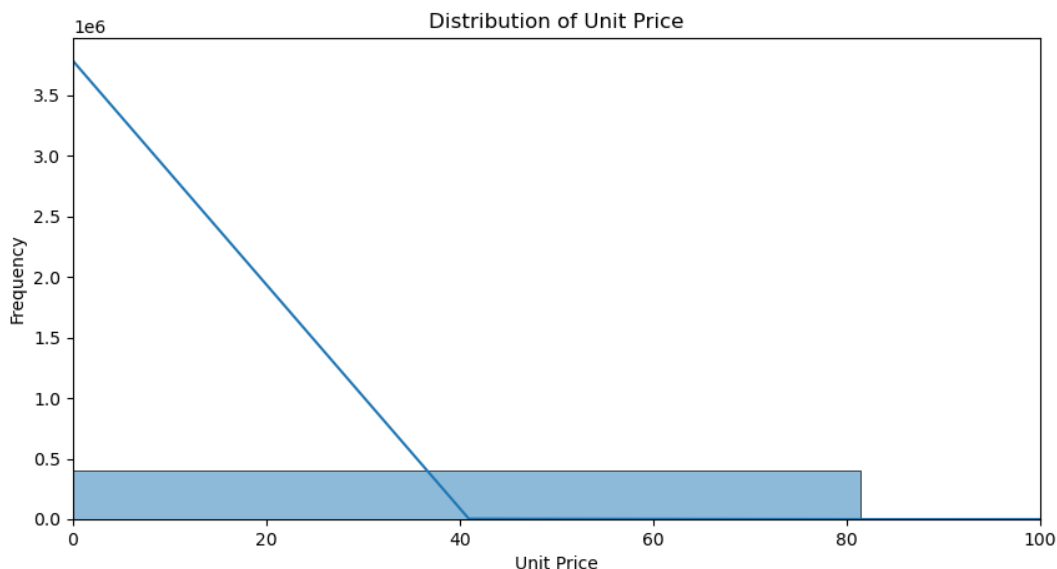
2.6 What are some of the descriptive statistics for key variables?

The key variables in the data set are Quantity and unit price and outliers are found in these variables as well.

Unit Price

- **Mean: 3.50** : This suggests that the average product sold is relatively low-cost, indicating the business sells affordable, fast-moving items.
- **Median: 2.08** : The median being significantly lower than the mean indicates right-skewness a few high-priced items are pulling the average up.
- **Standard Deviation: 9.79** : A large standard deviation relative to the mean implies high variability in pricing, possibly due to different product types e.g. bulk packs vs. single items).

- **Skewness: 186.51, Kurtosis: 59,005.72** : Extremely high values indicate strong right skew and heavy tails outliers or rare, expensive items distort the price distribution.



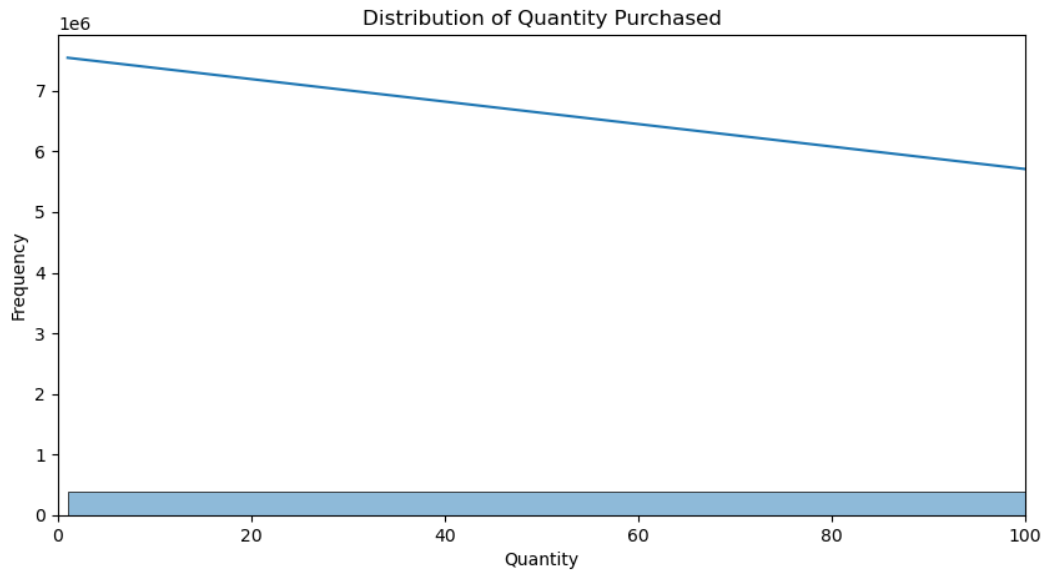
Python code to Visualize Unit Price Distribution-

```
import matplotlib.pyplot as plt

# Unit Price Distribution
plt.figure(figsize=(10, 5))
sns.histplot(df_clean['UnitPrice'], bins=100, kde=True)
plt.title('Distribution of Unit Price')
plt.xlabel('Unit Price')
plt.ylabel('Frequency')
plt.show()
```

Quantity

- **Mean: 12.06 units** : On average, customers purchase around 12 items per transaction.
- **Median: 5 units** : Again, the mean > median suggests a few very large orders inflating the average.
- **Standard Deviation: 42.83** : Indicates wide variation in order size likely driven by wholesale vs. retail buyers.
- **Skewness: 0.18** : Slight right skew, but symmetrical after removing returns.
- **Kurtosis: 94,317.56** : Extremely heavy tails a few huge orders create long tails.



Python code to Visualize Quantity Distribution-

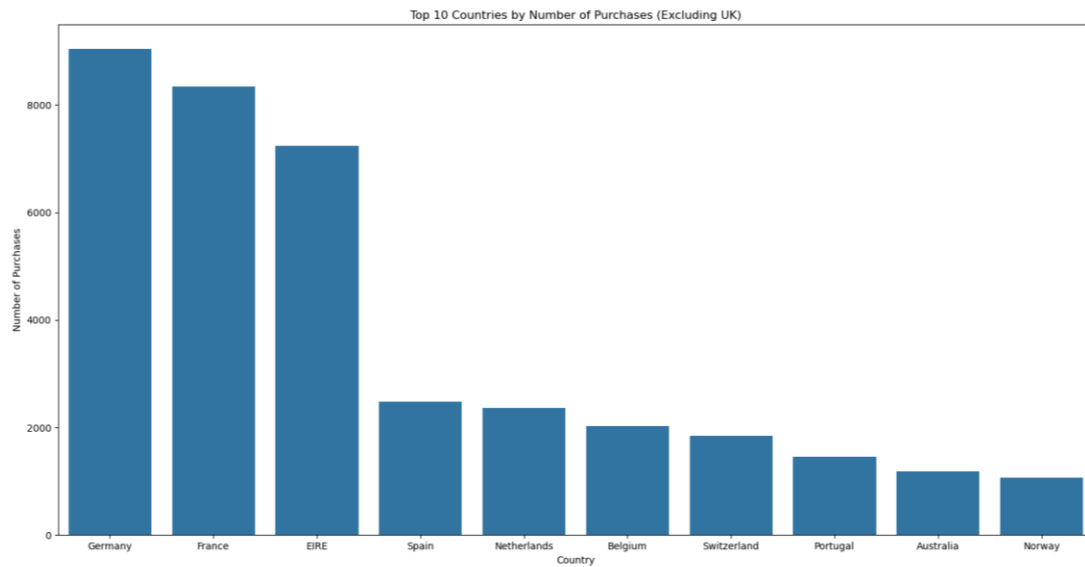
```
# 1. Quantity Distribution

plt.figure(figsize=(10, 5))
sns.histplot(df_clean['Quantity'], bins=100, kde=True)
plt.title('Distribution of Quantity Purchased')
plt.xlabel('Quantity')
plt.ylabel('Frequency')
plt.show()
```

Sales by Country

We have found that the UK dominates sales because most of the sales are from the UK, which only means those transactions are domestic. That is I have decided to analyze Top 10 countries by sales excluding the UK so that the company can know which countries are buying most products. Based on the data, the company can run marketing campaigns in those areas.

The bar chart shows the top 10 countries by sales among which Germany, France and Eire is leading. These countries can be potential international market for business. This chart will help business for internation expansion and overall sales growth.



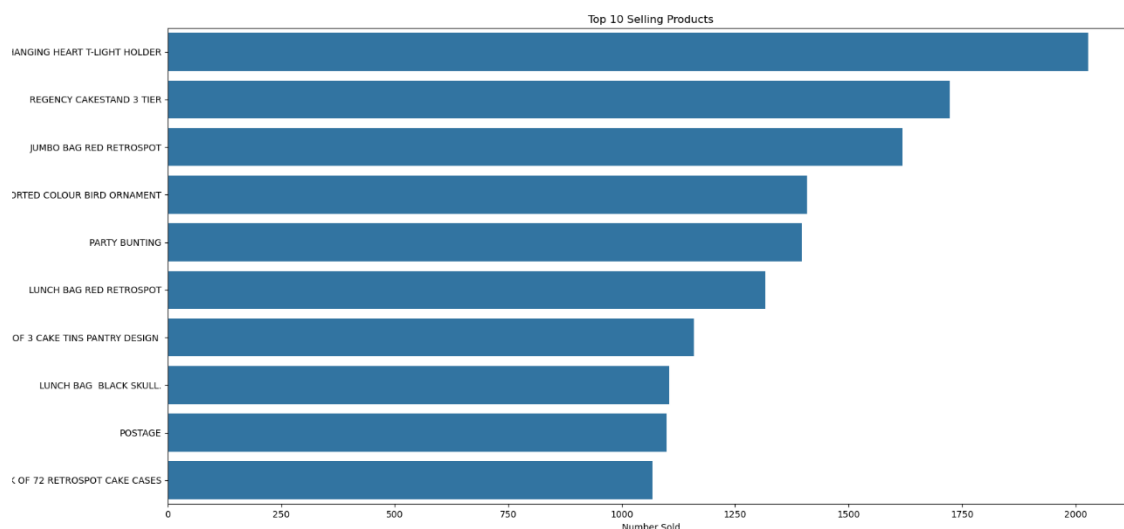
Python code to Visualize Top 10 Countries except UK-

```
##Top 10 countries by sales excluding UK

top_countries = df_clean[df_clean['Country'] != 'United
Kingdom']['Country'].value_counts().head(10)
sns.barplot(x=top_countries.index, y=top_countries.values)
plt.title('Top 10 Countries by Number of Purchases (Excluding UK)')
plt.xlabel('Country')
plt.ylabel('Number of Purchases')
plt.show()
```

Top Products by Sales

From the data in can be seen that the company has a lot of inventory, but all items are well performing. I have tried to gather the top products category based on sales to help business stock those items more and help business with inventory planning. From Visualization, the customers/vendors mostly purchase these following items. This data will also help the marketing team to determine which items to focus on for marketing campaigns.



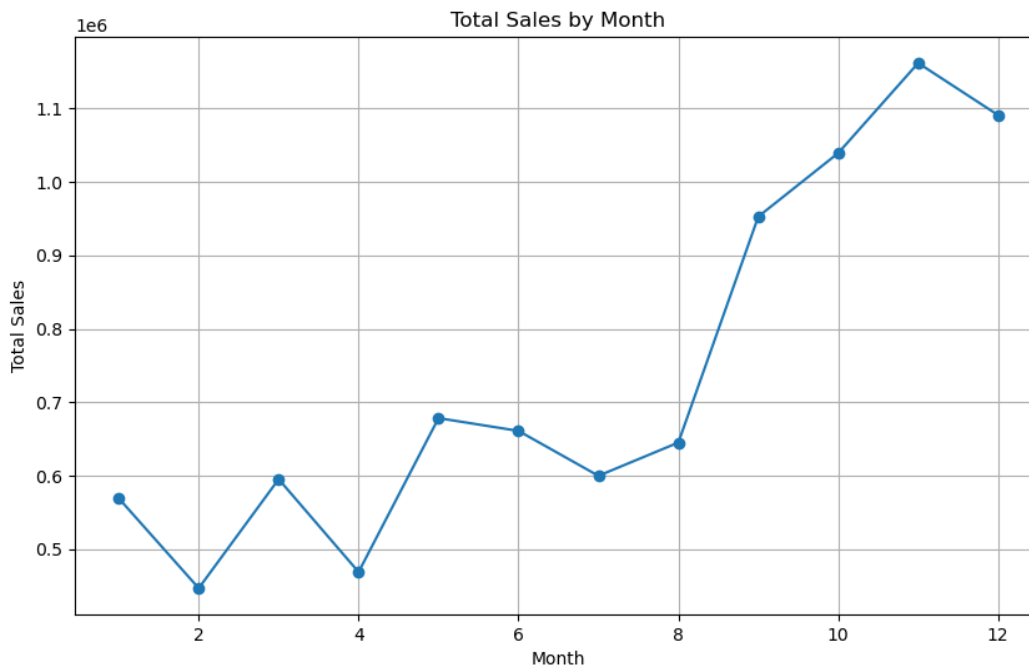
Python code to Visualize Top 10 Products –

```
## Top 10 Products by sales

top_products = df_clean['Description'].value_counts().head(10)
sns.barplot(x=top_products.values, y=top_products.index)
plt.title('Top 10 Selling Products')
plt.xlabel('Number Sold')
plt.ylabel('Product')
plt.show()
```

Total Sales by Month

Analyzing total sales on a monthly basis helps identify trends, seasonality, and sales performance over time. Once we group the data by month using the InvoiceDate field and compute total sales (calculated as Quantity * UnitPrice), we can observe the lowest sales and highest sales month. We can know the sales fluctuation and sales trend over the month and year by analyzing data. The dataset has date and time in a single column. At first, I separated the date and time. And then sorted the sales based on month. This data will help the business to know the seasonal trend, business peak and off-peak season, sales trend over period. These data will later help the business to determine for potential stockouts or the need for extra staffing in peak periods. It will help to plan inventory and logistics accordingly. The timeline will help align marketing campaigns with high-opportunity months and help investigate potential causes. The plot shows that the sales increased during October-December which means the holiday seasons Peak season for the business.



Python code to Visualize Total Sales by Month–

```
# Create new columns
df_clean['Invoice_Date'] = df_clean['InvoiceDate'].dt.date      # Only
Date
df_clean['InvoiceTime'] = df_clean['InvoiceDate'].dt.time      # Only
Time
df_clean['Month'] = df_clean['InvoiceDate'].dt.month    ##Only Month

#Deleting Original InvoiceDate column
df_clean.drop(columns=['InvoiceDate'], inplace=True)

##Calculating Total Sales
df_clean['TotalCost'] = df_clean['Quantity'] * df_clean['UnitPrice']

##Sales my month
sales_by_month = df_clean.groupby('Month')['TotalCost'].sum()

# Plot
plt.figure(figsize=(10,6))
sales_by_month.plot(kind='line', marker='o')
plt.title('Total Sales by Month')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.grid()
plt.show()
```

4.1 What did you learn about the data?

Through this analysis, I learned that the Online Retail dataset holds insights into customer conduct, sales patterns, and seasonal cycles. For example, most of the sales occur during working hours and are at their peak in November and December, which would reflect high seasonality and potentially holiday shopping highs. I also learned that data quality issues such as negative quantities and missing values can potentially have a significant impact on the analysis and need to be addressed with utmost caution.

This exercise sharpened several of my data analytics skills:

- **Data Cleaning:** I practiced detecting and handling missing values, invalid entries, and outliers.
- **Feature Engineering:** Extracting date parts like date, time, month helped me derive meaningful insights from a datetime variable.
- **Descriptive Statistics:** I learned how to interpret skewness, kurtosis, and distribution shapes instead of just reporting averages.
- **Visualization:** Using tools like seaborn and matplotlib allowed me to effectively communicate findings through visual trends.
- **Business Insight:** I practiced translating raw numbers into actionable business recommendations, such as optimal sales timing and inventory planning.

The biggest challenge for me was to convert my ideas into coding. I had so many ideas of how to analyze and visualize data. But due to limited knowledge of python coding and lack of competency, I had trouble organizing my thought into python coding. Moreover, there are so many function that I can use, but I didn't know which to use where.

- I would perform customer segmentation to understand behavior at a more granular level.
- I'd use time series models to forecast future sales.
- I'd explore product category performance to guide marketing or bundling strategies.

4.2 Why is or isn't Python a good tool to use for this type of analysis?

- Python is very handy for exploratory data analysis due to libraries like pandas, seaborn, matplotlib, and NumPy.
- Flexible and reproducible, sustaining scalable workflows.
- Well-documented, with abundant documentation and community support.

References

- Groner, D. H. (2022). Python for Data & Analytics: A Business-Oriented Approach. Prospect Press.
<https://akademos.vitalsource.com/books/9781943153985>
- Lichman, M. (2013). *Online Retail Data Set*. UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences.
<https://archive.ics.uci.edu/dataset/352/online+retail>